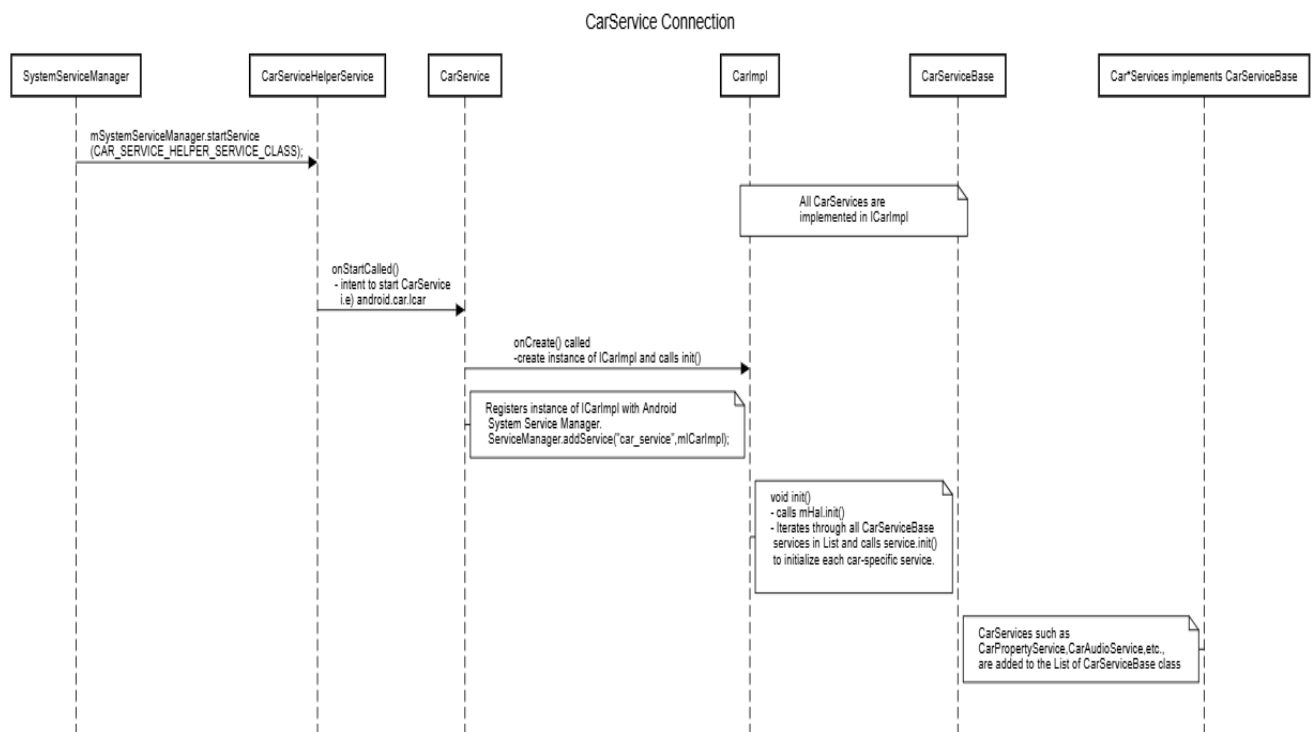


CarService Connection

In Android Automotive, **CarService** is the central service responsible for managing all automotive-related system services such as CarProperty, CarAudio, CarPower, and others.

The Android framework initializes **CarService** during the boot sequence, ensuring that automotive services are brought up and exposed to apps through the **ICar binder interface**.

1. Sequence diagram of CarService connection



1.SystemServer

- [SystemServer.java](#) is the main entry point for starting all Android framework services.
- During boot, it initializes core components and then starts services using **SystemServiceManager**.
- **Example:** `mSystemServiceManager = new SystemServiceManager(mSystemContext); startBootstrapServices(); startCoreServices(); startOtherServices();`
- **SystemServer** doesn't directly start each service class—it delegates the actual creation and lifecycle management to **SystemServiceManager**

2. SystemServiceManager

- Located in `frameworks/base/services/core/java/com/android/server/SystemServiceManager.java`.
- Responsible for instantiating SystemService classes.
- It checks if the device supports `FEATURE_AUTOMOTIVE`. If yes, it starts `CarServiceHelperService`.

Example :

```
if (hasSystemFeature(PackageManager.FEATURE_AUTOMOTIVE))
{ startService(CAR_SERVICE_HELPER_SERVICE); }
```

3. CarServiceHelperService

- Located in `com.android.internal.car.CarServiceHelperService`.
- Extends `SystemService`.
- Its role is to act as a **bridge** between the Android framework and the `CarService` running as a separate service.
- In `onStart()`, it binds to the `CarService` using the binder interface `android.car.ICar`.

```
@Override
public void onStart() {
    EventLog.writeEvent(EventLogTags.CAR_HELPER_START);

    IntentFilter filter = new IntentFilter(Intent.ACTION_REBOOT);
    filter.addAction(Intent.ACTION_SHUTDOWN);
    mContext.registerReceiverForAllUsers(mShutdownEventReceiver, filter, broadcastPermission.T
    mCarWatchdogDaemonHelper.addOnConnectionChangeListener(mConnectionListener);
    mCarWatchdogDaemonHelper.connect();
    Intent intent = new Intent();
    intent.setPackage("com.android.car");
    intent.setAction(CAR_SERVICE_INTERFACE);
    if (!mContext.bindServiceAsUser(intent, mCarServiceConnection, Context.BIND_AUTO_CREATE
        mHandler, UserHandle.SYSTEM)) {
        Slogf.wtf(TAG, msg: "cannot start car service");
    }
    loadNativeLibrary();
}
```

👉 Key Point:

This service separates the **normal Android framework** from the **automotive stack**. Normal system services talk to cars only through this helper.

4. CarService

- Found in `packages/services/Car/service/src/com/android/car/CarService.java`.
- Extends Service.
- The CarService (implemented in CarServiceImpl) is the central service in Android Automotive that manages all car-related features. When the service starts, its `onCreate()` method performs several important initialization steps:
 - ✓ Instantiates ICarImpl, the binder interface exposed as car_service and passes required contexts and system interface objects.
 - ✓ Initialize Car Sub-services by calling `mICarImpl.init()`.
 - ✓ `ServiceManager.addService("car_service", mICarImpl);`
This registers the ICarImpl instance with the Android system service manager under the name "car_service". After this, any client (CarServiceHelperService or apps using CarApi) can connect to the car_service via the binder IPC (**ICar interface**).

CarServiceImpl.java

```
mICarImpl = new ICarImpl(this,
    getBuiltinPackageContext(),
    mVehicle,
    SystemInterface.Builder.defaultSystemInterface(this).build(),
    mVehicleInterfaceName);
mICarImpl.init();

mVehicle.linkToDeath(mVehicleDeathRecipient);

ServiceManagerHelper.addService("car_service", mICarImpl);
SystemPropertiesHelper.set("boot.car_service_created", "1");
```

- CarService creates an ICarImpl object, passing the HAL binder (mVehicle) into it:

`mICarImpl = new ICarImpl(this, context, mVehicle, systemInterface, mVehicleInterfaceName);`

This step bridges CarService ↔ VHAL. Now ICarImpl has a handle to communicate with VHAL.

5. ICarImpl

- Implements the **ICar AIDL interface** (`android.car.ICar`).
- Creates instances of all Car services and is responsible for managing all car-related sub-services.
- Holds: `private final List<CarServiceBase> mServices = new ArrayList<>();`
- Adds them to the list: `mServices.add(mCarPropertyService);`
`mServices.add(mCarAudioService); mServices.add(mCarPowerManagementService);`

- Calls `init()` on each service so they are fully ready.

ICarImpl.java

```
@MainThread
void init() {
    mHal.init();
    t.traceBegin("CarService.initAllServices");
    for (CarServiceBase service : mAllServices) {
        t.traceBegin(service.getClass().getSimpleName());
        service.init();
    }
}
```

Two key points from the `init()` method in `ICarImpl`:

1. **Vehicle HAL Initialization:** `mHal.init()` sets up the hardware abstraction layer for vehicle communication.
2. **Service Initialization Loop:** Iterates through all `CarServiceBase` services in `mAllServices` and calls `service.init()` to initialize each car-specific service.

6. CarServiceBase

- Abstract base class for all automotive services.
- Defines lifecycle methods:
 - `init()` → setup logic.
 - `release()` → cleanup logic.
- Examples: `CarPropertyService`, `CarAudioService`, `CarBluetoothService`, `CarPowerManagementService`, etc., these classes implements `CarServiceBase` class and override its lifecycle methods in them.

CarServiceBase.java

```
public interface CarServiceBase {

    /**
     * service is started. All necessary initialization should be done and service should be
     * functional after this.
     */
    void init();

    /** service should stop and all resources should be released. */
    void release();

    /** Dumps its state. */
    void dump(IndentingPrintWriter writer);
}
```

7. Sub-Services Initialize Using VHAL

- `mICarImpl.init()` is called, which creates services such as:
- `CarPropertyService` → Talks to VHAL for vehicle properties (gear, ignition, speed).
- `CarPowerService` → Uses VHAL for power state and ignition control.
- `CarAudioService`, `CarPackageManagerService`, etc.

Conclusion:

`CarService` establishes a functional automotive system layer that allows applications and framework components to seamlessly access vehicle features. In short, **`CarService.onCreate()`** lays the foundation for the entire automotive service ecosystem in Android.

By looking at the **CarService connection in detail**, we see how Android Automotive extends the normal Android framework by introducing an isolated, binder-based service layer. This ensures that all car-related features (power, audio, property, etc.) are cleanly managed and accessed only through the **ICar interface**.

